

Apuntes DBA — Parte 2

Bases de Datos Nivel Senior / Staff

Query Planner · HA · Sharding · PITR · Event Sourcing ·
Seguridad · Monitoring

Aldo Zetina Muciño

zetinaa3@gmail.com

Índice general

1. Query Planner Internals	3
1.1. ¿Qué hace el Query Planner?	3
1.2. El modelo de costos	3
1.3. Estadísticas: pg_stats y ANALYZE	4
1.4. EXPLAIN y EXPLAIN ANALYZE	5
1.5. Métodos de acceso a datos	6
1.6. Tipos de Join	6
2. Performance Tuning Avanzado	8
2.1. Autovacuum: el guardián silencioso	8
2.2. Table Bloat y fill_factor	9
2.3. Connection Pooling con pgBouncer	10
2.4. Índices Avanzados	11
2.5. work_mem y shared_buffers	12
3. High Availability y Replicación	14
3.1. Streaming Replication	14
3.2. Replicación Síncrona vs Asíncrona	15
3.3. Patroni: HA Automatizado	16
3.4. Failover y Split-Brain	17
4. Sharding y Distribución	19
4.1. Cuando escalar horizontalmente	19
4.2. Partitioning nativo de PostgreSQL	19
4.3. Citus: Sharding sobre PostgreSQL	20
4.4. Foreign Data Wrappers (FDW)	21
4.5. CockroachDB y Vitess: alternativas	22
5. Migraciones en Producción	23
5.1. El problema de las migraciones en producción	23
5.2. Expand-Contract Pattern	23
5.3. CREATE INDEX CONCURRENTLY	24
5.4. pg_repack: reorganizar sin bloquear	25
5.5. Flyway y Liquibase	25
6. PITR, WAL y Backup	28
6.1. Write-Ahead Log en profundidad	28
6.2. WAL Archiving	29
6.3. pgBackRest: backup empresarial	29
6.4. Procedimiento PITR	30
7. Event Sourcing y CQRS	33
7.1. Event Sourcing: estado como historia	33

7.2. Event Store con PostgreSQL	33
7.3. CQRS: separar lectura de escritura	34
7.4. Outbox Pattern	35
7.5. Replicación Lógica para CDC	36
8. Seguridad en PostgreSQL	38
8.1. Row-Level Security (RLS)	38
8.2. Cifrado con pgcrypto	39
8.3. Autenticación y pg_hba.conf	40
8.4. Principio de Mínimo Privilegio	41
8.5. pg_audit: logging de queries	41
9. Monitoring y Observabilidad	43
9.1. pg_stat_statements: queries más costosas	43
9.2. Vistas de sistema esenciales	44
9.3. pgBadger: análisis de logs	45
9.4. Prometheus + postgres_exporter	45
9.5. Dashboards Grafana esenciales	47
10. Preguntas de Entrevista Senior/Staff	48
10.1. Diseño de sistemas	48
10.2. Performance y troubleshooting	49
10.3. High Availability y recuperación	49
10.4. Preguntas de fundamentos a nivel Staff	50
10.5. Cheat sheet de respuestas clave	50

Capítulo 1

Query Planner Internals

1.1. ¿Qué hace el Query Planner?

El Query Planner (o Query Optimizer) es el componente de PostgreSQL que toma una consulta SQL y decide *cómo* ejecutarla físicamente. No importa si el SQL es correcto: si el planner elige un plan malo, la consulta puede tardar segundos en lugar de milisegundos. Entender el planner es la diferencia entre un DBA junior y uno senior.

Pipeline de ejecución de una query

1. **Parser:** SQL texto → árbol sintáctico (parse tree)
2. **Analyzer/Rewriter:** valida objetos (tablas, columnas existen), aplica reglas (vistas → subqueries)
3. **Planner/Optimizer:** genera planes posibles, estima costos, elige el más barato
4. **Executor:** ejecuta el plan elegido contra los datos reales

El planner **nunca ejecuta** nada; solo genera y evalúa planes. El costo es una estimación, no un tiempo real.

1.2. El modelo de costos

PostgreSQL asigna un costo a cada operación del plan basado en parámetros de configuración. El costo no está en segundos; es una unidad abstracta donde `seq_page_cost = 1.0` es la referencia.

Parámetros clave de costo

- `seq_page_cost = 1.0` — costo de leer una página secuencialmente (de disco o buffer)
- `random_page_cost = 4.0` — costo de acceso aleatorio (salto de disco). En SSD: bajar a 1.1–1.5
- `cpu_tuple_cost = 0.01` — costo de procesar una fila
- `cpu_index_tuple_cost = 0.005` — costo de procesar una entrada de índice
- `cpu_operator_cost = 0.0025` — costo de evaluar un operador (`=`, `<`, etc.)

- `effective_cache_size` — estimación de cuánto RAM tiene el OS para caché de páginas. Afecta la decisión index scan vs seq scan

Listing 1.1: Ajustar costos para SSD

```

1  -- En postgresql.conf o por sesión:
2  SET random_page_cost = 1.1;
3  SET effective_cache_size = '12GB';  -- 75% de RAM total aprox
4
5  -- Verificar valores actuales:
6  SHOW random_page_cost;
7  SELECT name, setting, unit FROM pg_settings
8  WHERE name IN ('random_page_cost', 'seq_page_cost', '
    effective_cache_size');
```

1.3. Estadísticas: pg_stats y ANALYZE

El planner estima selectividad (¿cuántas filas retorna este filtro?) usando estadísticas recopiladas por ANALYZE. Sin estadísticas actualizadas, el planner genera estimaciones malas y elige planes subóptimos.

Listing 1.2: Explorar estadísticas del planner

```

1  -- Estadísticas por columna:
2  SELECT attname, n_distinct, correlation, most_common_vals,
    most_common_freqs
3  FROM pg_stats
4  WHERE tablename = 'orders' AND attname = 'status';
5
6  -- n_distinct > 0: número exacto de valores únicos
7  -- n_distinct < 0: fracción de filas únicas (ej: -0.95 = 95% únicos
    )
8  -- correlation: 1.0 = físicamente ordenado (index scan muy
    eficiente)
9  --                0.0 = aleatorio (index scan costoso, seq scan mejor
    )
10
11 -- Forzar recolección de estadísticas:
12 ANALYZE orders;
13 ANALYZE VERBOSE orders;  -- muestra progreso
14
15 -- Statistics target por columna (default 100, rango 1-10000):
16 ALTER TABLE orders ALTER COLUMN status SET STATISTICS 500;
17 ANALYZE orders;
18 -- Mayor target = mejores estimaciones en columnas con distribución
    sesgada
```

💡 Cuándo aumentar statistics target

Columna con distribución muy sesgada: pocos valores dominan (ej: `status = 'completed'` es 95% de las filas). Con target bajo, el planner no ve los valores raros y subestima su selectividad. Sube a 200-500 en columnas de filtros frecuentes con distribución irregular.

1.4. 📄 EXPLAIN y EXPLAIN ANALYZE

EXPLAIN muestra el plan sin ejecutar. EXPLAIN ANALYZE ejecuta y muestra tiempos reales. La diferencia entre filas estimadas y reales revela problemas de estadísticas.

Listing 1.3: Anatomía de EXPLAIN ANALYZE

```

1 EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
2 SELECT o.id, c.name, SUM(i.amount)
3 FROM orders o
4 JOIN customers c ON c.id = o.customer_id
5 JOIN items i ON i.order_id = o.id
6 WHERE o.created_at > NOW() - INTERVAL '30 days'
7 GROUP BY o.id, c.name;
8
9 -- Salida típica:
10 -- HashAggregate (cost=1234.56..1289.78 rows=500 width=52)
11 --      (actual time=45.2..47.8 rows=423 loops=1)
12 --      Buffers: shared hit=890 read=234
13 --      -> Hash Join (cost=...) (actual time=... rows=...)
14 --          Hash Cond: (i.order_id = o.id)
15 --          -> Seq Scan on items (cost=...) (actual rows=12847)
16 --          -> Hash (actual rows=423 loops=1)
17 --              -> Index Scan on orders using idx_orders_created_at
18 --                  Index Cond: (created_at > ...)
19
20 -- Interpretar:
21 -- cost=inicio..total -      estimación del planner
22 -- actual time=.. -      tiempo real en ms
23 -- rows=N (estimado) vs rows=N (actual) → si difieren mucho: stats
24 --                               problema
25 -- Buffers: shared hit=X read=Y → hit=de cache, read=de disco
26 -- loops=N → si > 1: este nodo se repite N veces (nested loop)

```

Listing 1.4: Detectar estimaciones malas

```

1 -- Si "rows estimado" difiere 10x+ de "rows actual": problema de
2 --   stats
3 -- Ejemplo de plan malo:
4 -- Seq Scan on orders (cost=0..50000 rows=5 width=...)
5 --      (actual time=... rows=45000 ...)
6 -- El planner creyó que había 5 filas, había 45000 → eligió seq
7 --   scan "barato"
8 -- pero en realidad debía haber filtrado antes.

```

```

7
8 -- Herramienta externa: https://explain.dalibo.com (visual)
9 -- Pega el output de EXPLAIN (FORMAT JSON) para visualización
10 EXPLAIN (ANALYZE, FORMAT JSON)
11 SELECT * FROM orders WHERE customer_id = 42;

```

1.5. 📁 Métodos de acceso a datos

🧠 Seq Scan vs Index Scan vs Index Only Scan

- **Seq Scan:** lee cada página de la tabla en orden. Costo fijo por página. Mejor cuando se leen $>20-30\%$ de las filas.
- **Index Scan:** usa el índice para encontrar TIDs (row pointers), luego hace heap fetch por cada TID. Ideal para baja selectividad ($<5\%$ de filas).
- **Index Only Scan:** todas las columnas requeridas están en el índice. No hay heap fetch. Más rápido que Index Scan pero requiere visibility map actualizado (VACUUM frecuente).
- **Bitmap Index Scan:** acumula TIDs del índice en un bitmap, luego hace heap fetches ordenados por bloque físico. Reduce I/O aleatorio. Aparece cuando el planner espera muchos hits de índice.

Listing 1.5: Forcing plan methods (debugging solamente)

```

1 -- Deshabilitar métodos para comparar planes:
2 SET enable_seqscan = OFF;
3 SET enable_hashjoin = OFF;
4 SET enable_mergejoin = OFF;
5
6 -- Ver el plan alternativo:
7 EXPLAIN SELECT * FROM orders WHERE status = 'pending';
8
9 -- IMPORTANTE: restaurar siempre después de diagnosticar
10 RESET enable_seqscan;
11 RESET enable_hashjoin;

```

1.6. 🔗 Tipos de Join

🧠 Nested Loop vs Hash Join vs Merge Join

- **Nested Loop:** para cada fila del outer, busca en el inner. Costo $O(N \times M)$ sin índice. Con índice en inner: $O(N \times \log M)$. Ideal cuando outer es pequeño y inner tiene índice.
- **Hash Join:** construye hash table del inner (el más pequeño), luego escanea outer. Costo $O(N + M)$. Ideal para joins grandes sin índice. Requiere `work_mem` para el hash table.

- **Merge Join:** ambas relaciones ordenadas $\rightarrow$ un solo pasado. Costo $O(N \log N + M \log M)$ si hay que ordenar. Ideal cuando ambas entradas ya vienen ordenadas (por índice o SORT previo).

Listing 1.6: work_mem y Hash Join

```

1  -- Si hash join usa disco (spill), aumentar work_mem:
2  SET work_mem = '256MB';
3
4  -- Verificar si hubo spill:
5  EXPLAIN (ANALYZE, BUFFERS)
6  SELECT * FROM big_table a JOIN other_big b ON a.id = b.id;
7  -- Buscar: "Batches: 4"  $\rightarrow$  hubo spill a disco (ideal: Batches: 1)
8
9  -- CUIDADO: work_mem es por operación, por conexión.
10 -- 100 conexiones  $\times$  4 sort ops  $\times$  256MB = 102 GB RAM posible
11 -- Ajustar globalmente con cuidado; preferir por sesión/query

```



Preguntas típicas

1. ¿Qué información usa el Query Planner para estimar el costo de una query?
2. ¿Cuándo el planner elige un Seq Scan sobre un Index Scan aunque exista el índice?
3. ¿Qué significa que las filas estimadas y reales difieran 100x en EXPLAIN ANALYZE?
4. Explica la diferencia entre Index Scan e Index Only Scan. ¿Qué se necesita para que el segundo funcione eficientemente?
5. ¿Por qué random_page_cost es clave en servidores con SSD y cómo lo ajustarías?
6. ¿Qué hace ANALYZE y cuándo se debe correr manualmente?

Capítulo 2

⚡ Performance Tuning Avanzado

2.1. 🧹 Autovacuum: el guardián silencioso

MVCC (Multi-Version Concurrency Control) significa que cada UPDATE en PostgreSQL crea una nueva versión de la fila; la vieja queda como “dead tuple”. Autovacuum limpia dead tuples, actualiza estadísticas y previene **transaction ID wraparound** (el peor incidente de PostgreSQL: la BD se pone en modo solo-lectura).

🧠 Cómo funciona autovacuum

- **autovacuum launcher:** proceso maestro, activa workers según tablas que lo necesitan
- **autovacuum worker:** trabaja una tabla a la vez; limitado por `autovacuum_max_workers` (default 3)
- **Trigger de vacuum:** tabla con $> \text{autovacuum_vacuum_threshold} + \text{autovacuum_vacuum_scale_factor} \times \text{n_live_tup}$ dead tuples
- **Trigger de analyze:** similar pero para estadísticas (`autovacuum_analyze_scale_factor`)
- Autovacuum usa `cost delay` para no saturar I/O: trabaja en ráfagas, luego duerme

Listing 2.1: Diagnosticar estado de autovacuum

```
1  -- Tablas que más necesitan vacuum:
2  SELECT schemaname, relname,
3         n_dead_tup, n_live_tup,
4         round(n_dead_tup::numeric / NULLIF(n_live_tup + n_dead_tup,
5         0) * 100, 2) AS dead_pct,
6         last_autovacuum, last_autoanalyze,
7         autovacuum_count, autoanalyze_count
8  FROM pg_stat_user_tables
9  ORDER BY n_dead_tup DESC
10 LIMIT 20;
11
12 -- Vacuums activos ahora mismo:
13 SELECT pid, query, state, now() - pg_stat_activity.query_start AS
14        duration
15 FROM pg_stat_activity
16 WHERE query LIKE 'autovacuum%';
17
18 -- ¿Hay tablas con bloat alto?
```

```

17 SELECT relname, pg_size_pretty(pg_total_relation_size(oid)) AS
    total_size,
18        pg_size_pretty(pg_relation_size(oid)) AS table_size
19 FROM pg_class WHERE relkind = 'r'
20 ORDER BY pg_total_relation_size(oid) DESC LIMIT 10;

```

Listing 2.2: Tuning de autovacuum por tabla

```

1  -- Para tablas con alta tasa de cambios (ej: logs, eventos):
2  ALTER TABLE events SET (
3      autovacuum_vacuum_scale_factor = 0.01,      -- vacuum cuando 1% de
        filas son dead
4      autovacuum_analyze_scale_factor = 0.005,
5      autovacuum_vacuum_cost_delay = 2,           -- ms de delay entre
        ráfagas (default 20)
6      autovacuum_vacuum_cost_limit = 400          -- trabajo por ráfaga (
        default 200)
7  );
8
9  -- VACUUM manual agresivo (para emergencias, bloquea concurrent
        vacuums):
10 VACUUM (ANALYZE, VERBOSE) orders;
11
12 -- VACUUM FULL: compacta tabla reescribiéndola (exclusivo, bloquea
        todo):
13 -- Solo en ventana de mantenimiento, o usar pg_repack
14 VACUUM FULL orders;

```

2.2. Table Bloat y fill_factor

Bloat ocurre cuando vacuum no puede reclamar espacio porque hay transacciones largas activas, o cuando el espacio muerto no se reutiliza por la estructura física de páginas.

Listing 2.3: fill_factor para tablas con muchos UPDATES

```

1  -- fill_factor: qué porcentaje de cada página se llena al INSERT.
2  -- El resto se reserva para UPDATES "in-page" (HOT updates).
3  -- HOT update: si la fila cabe en la misma página → no se actualiza
        el índice → mucho más rápido
4
5  -- Para tablas con muchos UPDATES, reducir fill_factor:
6  ALTER TABLE orders SET (fillfactor = 70);
7  -- Necesita VACUUM o CLUSTER para aplicar en páginas existentes
8
9  -- Para tablas append-only (logs, eventos):
10 ALTER TABLE audit_log SET (fillfactor = 100); -- default
11
12 -- Ver fill_factor actual:
13 SELECT relname, reloptions FROM pg_class
14 WHERE relname = 'orders';

```

2.3. Connection Pooling con pgBouncer

Cada conexión de PostgreSQL es un proceso OS. Con 200+ conexiones directas, el overhead de context switching degrada el rendimiento. pgBouncer es un proxy que multiplexa muchas conexiones de cliente sobre pocas conexiones reales.

Modos de pgBouncer

- **Session mode:** una conexión de PG por conexión de cliente mientras dure la sesión. Seguro pero no multiplexea mucho.
- **Transaction mode:** una conexión de PG por transacción. Multiplexeo real. **Limitación:** no se pueden usar prepared statements de sesión, SET LOCAL, LISTEN/NOTIFY.
- **Statement mode:** una conexión por statement. Solo para apps que no usan transacciones. Raramente útil.

Listing 2.4: pgbouncer.ini — configuración típica

```

1 [databases]
2 mydb = host=127.0.0.1 port=5432 dbname=mydb
3
4 [pgbouncer]
5 listen_port = 6432
6 listen_addr = *
7 auth_type = scram-sha-256
8 auth_file = /etc/pgbouncer/userlist.txt
9
10 pool_mode = transaction
11 max_client_conn = 1000
12 default_pool_size = 20           ; conexiones reales a PG
13 min_pool_size = 5
14 reserve_pool_size = 5
15 reserve_pool_timeout = 3
16
17 server_idle_timeout = 600
18 client_idle_timeout = 0
19
20 ; Stats cada 60s en pgbouncer log
21 stats_period = 60

```

Listing 2.5: Monitorear pgBouncer desde psql

```

1 -- Conectarse a la base admin de pgbouncer (puerto 6432):
2 -- psql -h localhost -p 6432 -U pgbouncer pgbouncer
3
4 SHOW POOLS;      -- ver cl_active, cl_waiting, sv_active, sv_idle por
   pool
5 SHOW STATS;      -- requests/s, bytes/s, query time promedio
6 SHOW CLIENTS;    -- conexiones de cliente activas
7 SHOW SERVERS;    -- conexiones a PostgreSQL activas

```

2.4. Índices Avanzados

Listing 2.6: Partial, expression y covering indexes

```

1  -- Partial index: solo indexa las filas que cumplen la condición
2  -- Mucho más pequeño y mantenido solo cuando aplica el WHERE
3  CREATE INDEX idx_orders_pending
4  ON orders (created_at)
5  WHERE status = 'pending';
6
7  -- Expression index: indexa el resultado de una expresión
8  CREATE INDEX idx_users_email_lower
9  ON users (lower(email));
10
11 -- Ahora solo usa el índice con: WHERE lower(email) = 'foo@example.
    com'
12 -- No con: WHERE email = 'foo@example.com' (string diferente)
13
14 -- Covering index (INCLUDE): añade columnas al índice sin que sean
    clave
15 -- Permite Index Only Scan sin ir al heap
16 CREATE INDEX idx_orders_customer_covering
17 ON orders (customer_id) INCLUDE (status, total_amount, created_at);
18
19 -- La query: SELECT status, total_amount FROM orders WHERE
    customer_id = 42
20 -- puede usar Index Only Scan si visibility map está al día
21
22 -- Concurrent index build (no bloquea escrituras):
23 CREATE INDEX CONCURRENTLY idx_items_order_id ON items (order_id);
24 -- Tarda más, pero la tabla sigue accesible

```

Listing 2.7: Detectar índices ineficientes

```

1  -- Índices nunca usados:
2  SELECT schemaname, relname, indexrelname, idx_scan, idx_tup_read
3  FROM pg_stat_user_indexes
4  WHERE idx_scan = 0
5  ORDER BY pg_relation_size(indexrelid) DESC;
6
7  -- Tablas con seq scans frecuentes (posibles índices faltantes):
8  SELECT relname, seq_scan, idx_scan,
9         seq_scan::float / NULLIF(seq_scan + idx_scan, 0) AS
           seq_ratio
10 FROM pg_stat_user_tables
11 WHERE seq_scan > 100
12 ORDER BY seq_scan DESC;
13
14 -- Tamaño de índices:
15 SELECT relname AS table, indexrelname AS index,
16        pg_size_pretty(pg_relation_size(indexrelid)) AS size
17 FROM pg_stat_user_indexes

```

```
18 ORDER BY pg_relation_size(indexrelid) DESC;
```

2.5. work_mem y shared_buffers

Parámetros de memoria críticos

- **shared_buffers:** caché de páginas de PG. Regla: 25–40 % de RAM. No poner >40 % porque el OS cache también ayuda y PG lo usa.
- **work_mem:** RAM por operación de sort/hash. Multiplicado por número de sorts simultáneos × conexiones. Regla conservadora: RAM / max_connections / 4.
- **maintenance_work_mem:** para VACUUM, CREATE INDEX, CLUSTER. Puede ser mayor (512MB–1GB en servidores grandes). No afecta conexiones normales.
- **wal_buffers:** buffer para escritura de WAL. Default: 3 % de shared_buffers (mínimo 32kB, máximo 64MB). Raramente necesita ajuste.
- **effective_cache_size:** no reserva RAM; solo le dice al planner cuánta RAM total (PG + OS) puede usarse como caché. Afecta decisiones de Index Scan. Poner 50–75 % de RAM total.

Listing 2.8: Configuración de memoria recomendada (servidor 32GB)

```
1 -- postgresql.conf:
2 shared_buffers = 8GB           -- 25% de 32GB
3 work_mem = 64MB               -- conservador; subir por sesión
   si necesario
4 maintenance_work_mem = 1GB
5 effective_cache_size = 24GB    -- 75% de 32GB
6 wal_buffers = 64MB
7
8 -- Para una query pesada específica (analytics), subir work_mem
   temporalmente:
9 SET work_mem = '2GB';
10 SELECT /* heavy aggregation */ ...;
11 RESET work_mem;
```

Preguntas típicas

1. ¿Qué es table bloat y cómo lo causas involuntariamente?
2. ¿Cuándo usarías pgBouncer en modo transaction vs session? ¿Qué pierdes en modo transaction?
3. Explica qué es un HOT update y cómo fill_factor lo facilita.
4. ¿Por qué un índice nunca usado sigue costando dinero en producción?
5. ¿Qué pasa si pones work_mem = 2GB con 200 conexiones y cada una hace 3 sorts simultáneos?

6. ¿Cómo detectas si autovacuum no está manteniendo una tabla?

Capítulo 3

High Availability y Replicación

3.1. Streaming Replication

PostgreSQL usa **WAL (Write-Ahead Log)** como fuente de verdad para replicación. El primary envía registros WAL al standby en tiempo real (streaming), que los aplica para mantener una copia casi idéntica.

Arquitectura de Streaming Replication

- **Primary:** acepta lecturas y escrituras. Genera WAL en `pg_wal/`.
- **WAL Sender:** proceso en primary que envía WAL al standby por TCP.
- **WAL Receiver:** proceso en standby que recibe WAL y lo escribe localmente.
- **Startup Process:** en standby, aplica WAL continuamente (recovery loop).
- **Replication lag:** delay entre cambio en primary y aplicación en standby. Medido en bytes (write lag) y tiempo (replay lag).

Listing 3.1: Configurar streaming replication (Primary)

```
1  -- postgresql.conf en PRIMARY:
2  wal_level = replica           -- mínimo para replicación
3  max_wal_senders = 5          -- procesos WAL sender simultáneos
4  wal_keep_size = 1GB         -- retener WAL para standbys lentos
5  hot_standby = on             -- standbys pueden servir lecturas (
    default on)
6
7  -- pg_hba.conf en PRIMARY (agregar):
8  -- host replication replicator 10.0.0.2/32 scram-sha-256
9
10 -- Crear usuario de replicación:
11 CREATE USER replicator WITH REPLICATION ENCRYPTED PASSWORD '
    securepass';
```

Listing 3.2: Configurar streaming replication (Standby)

```
1  -- Crear standby con pg_basebackup:
2  pg_basebackup -h primary_host -U replicator -D /var/lib/postgresql/
    data \
3      --wal-method=stream --checkpoint=fast --progress
4
5  -- postgresql.conf en STANDBY:
6  hot_standby = on
```

```

7 hot_standby_feedback = on          -- evita que autovacuum en primary
   cancele queries en standby
8
9 -- postgresql.conf o recovery.conf (PG 12+: primary_conninfo en
   postgresql.conf):
10 primary_conninfo = 'host=primary_host port=5432 user=replicator
   password=securepass'
11
12 -- Crear archivo de señal para modo standby (PG 12+):
13 touch /var/lib/postgresql/data/standby.signal

```

Listing 3.3: Monitorear replication lag

```

1 -- En el PRIMARY:
2 SELECT client_addr, state, sent_lsn, write_lsn, flush_lsn,
   replay_lsn,
3       write_lag, flush_lag, replay_lag,
4       sync_state
5 FROM pg_stat_replication;
6
7 -- replay_lag > 0: standby está por detrás
8 -- sync_state: 'async' (default), 'sync', 'quorum'
9
10 -- En el STANDBY:
11 SELECT now() - pg_last_xact_replay_timestamp() AS replication_delay
   ;
12 SELECT pg_is_in_recovery();                -- true si es standby
13 SELECT pg_last_wal_replay_lsn();           -- último WAL aplicado

```

3.2. Replicación Síncrona vs Asíncrona

Modos de commit

- **Asíncrona (default):** primary confirma commit sin esperar al standby. RPO > 0 (puede haber pérdida de datos). Latencia mínima.
- **Síncrona:** primary espera que el standby confirme haber recibido (y opcionalmente aplicado) el WAL antes de retornar el commit. RPO = 0 (sin pérdida). Latencia mayor.
- **synchronous_commit = remote_apply:** espera a que el standby *aplique* el WAL. Garantía más fuerte pero latencia máxima.

Listing 3.4: Configurar replicación síncrona

```

1 -- postgresql.conf en PRIMARY:
2 synchronous_standby_names = 'FIRST 1 (standby1, standby2)'
3 -- FIRST 1: el primero de la lista que esté disponible se vuelve
   síncrono
4 -- ANY 1: cualquiera de la lista
5

```



```

6 synchronous_commit = on          -- default; espera flush en standby
7 -- synchronous_commit = remote_apply -- más fuerte: espera
  aplicación
8
9 -- Por transacción (tradeoff rendimiento/durabilidad):
10 BEGIN;
11 SET LOCAL synchronous_commit = off; -- esta tx es asíncrona
12 INSERT INTO audit_log ...;
13 COMMIT;

```

3.3. Patroni: HA Automatizado

Patroni es un template de HA para PostgreSQL que usa un DCS (Distributed Configuration Store: etcd, Consul, ZooKeeper) para coordinar elecciones de líder y failover automático.

Componentes de Patroni

- **Patroni agent:** corre en cada nodo de PG. Monitorea el nodo, actualiza el DCS, maneja failover.
- **DCS (etcd):** almacena quién es el líder y el estado del clúster. El “árbitro” que evita split-brain.
- **HAProxy/pgBouncer:** balanceador de carga. Dirige escrituras al primary, lecturas a standbys. Patroni expone health endpoints HTTP para que HAProxy detecte el primary.
- **patronictl:** CLI para gestionar el clúster (switchover manual, failover, restart, reinitialización).

Listing 3.5: patroni.yml — configuración básica

```

1 scope: postgres-cluster
2 namespace: /db/
3 name: node1
4
5 restapi:
6   listen: 0.0.0.0:8008
7   connect_address: 10.0.0.1:8008
8
9 etcd:
10  hosts: 10.0.0.10:2379,10.0.0.11:2379,10.0.0.12:2379
11
12 bootstrap:
13   dcs:
14     ttl: 30
15     loop_wait: 10
16     retry_timeout: 30
17     maximum_lag_on_failover: 1048576 # 1MB lag máximo para ser
18     eligible

```

```

19  initdb:
20      - encoding: UTF8
21      - data-checksums
22
23  postgresql:
24      listen: 0.0.0.0:5432
25      connect_address: 10.0.0.1:5432
26      data_dir: /var/lib/postgresql/data
27      authentication:
28          replication:
29              username: replicator
30              password: securepass
31      superuser:
32          username: postgres
33          password: adminpass

```

Listing 3.6: patronictl — operaciones comunes

```

1  # Ver estado del clúster:
2  patronictl -c /etc/patroni.yml list
3
4  # Switchover manual (planned, sin downtime notable):
5  patronictl -c /etc/patroni.yml switchover --master node1 --
    candidate node2
6
7  # Failover forzado (emergencia):
8  patronictl -c /etc/patroni.yml failover --master node1 --force
9
10 # Reiniciar nodo:
11 patronictl -c /etc/patroni.yml restart postgres-cluster node1
12
13 # Reinicializar standby (después de desync):
14 patronictl -c /etc/patroni.yml reinit postgres-cluster node2

```

3.4. Failover y Split-Brain



Split-brain: dos nodos creen ser el primary simultáneamente. Ambos aceptan escrituras, los datos divergen, y no hay forma automática de reconciliar. Es el peor escenario en HA.

Patroni lo evita con el DCS: el primary tiene un “lease” (lock) en etcd. Si pierde conectividad con etcd, **se degrada a standby** aunque aún pueda hablar con clientes. El nodo que no puede adquirir el lock no puede ser primary, sin importar su estado de red con los otros nodos de PG.

Listing 3.7: Health endpoint de Patroni para HAProxy

```

1  -- Patroni expone en :8008:
2  -- GET /primary → 200 si es primary, 503 si no

```

```
3 -- GET /replica → 200 si es standby, 503 si no
4 -- GET /health → 200 siempre (para liveness check)
5
6 -- haproxy.cfg:
7 -- backend postgres_primary
8 --     option httpchk GET /primary
9 --     server node1 10.0.0.1:5432 check port 8008
10 --     server node2 10.0.0.2:5432 check port 8008
11 --     server node3 10.0.0.3:5432 check port 8008
```



Preguntas típicas

1. ¿Cuál es la diferencia entre RPO y RTO? ¿Cómo los afecta la replicación síncrona vs asíncrona?
2. Explica cómo Patroni evita split-brain.
3. ¿Qué es hot_standby_feedback y por qué importa en un clúster con queries largas en el standby?
4. ¿Cuándo usarías synchronous_commit = remote_apply en lugar de on?
5. Un standby tiene 5 minutos de lag. ¿Cuáles son las causas más probables y cómo diagnosticas?
6. ¿Qué ocurre si etcd no tiene quórum? ¿El clúster sigue funcionando?

Capítulo 4



Sharding y Distribución

4.1. Cuándo escalar horizontalmente

Escalar verticalmente (más CPU, RAM, SSD) es siempre la primera respuesta: más simple, sin cambios de arquitectura. El sharding horizontal se justifica cuando una sola máquina no puede manejar el volumen de escrituras o el tamaño de datos supera lo manejable en un nodo.



Límites que fuerzan la distribución

- **Write throughput:** un primary puede saturar su I/O de WAL. Sharding divide las escrituras entre múltiples primaries.
- **Dataset size:** terabytes que no caben en RAM de un nodo degradan el performance a I/O puro.
- **Geografía:** usuarios en distintas regiones necesitan latencia baja → datos cerca de ellos.
- **Compliance:** datos de ciertos países deben residir en esa jurisdicción.

Costo del sharding: queries cross-shard se vuelven joins distribuidos (caros), transacciones distribuidas (2PC = complejo), y el operacional se multiplica.

4.2. Partitioning nativo de PostgreSQL

El partitioning es el primer paso antes de sharding real. Divide una tabla lógica en múltiples tablas físicas (particiones) en el *mismo* nodo.

Listing 4.1: Table Partitioning por rango (tiempo)

```
1  -- Tabla madre particionada:
2  CREATE TABLE events (
3      id          BIGSERIAL,
4      user_id     BIGINT NOT NULL,
5      event_type  TEXT NOT NULL,
6      created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
7      payload     JSONB
8  ) PARTITION BY RANGE (created_at);
9
10 -- Particiones por mes:
11 CREATE TABLE events_2025_01 PARTITION OF events
12     FOR VALUES FROM ('2025-01-01') TO ('2025-02-01');
13
```

```

14 CREATE TABLE events_2025_02 PARTITION OF events
15     FOR VALUES FROM ('2025-02-01') TO ('2025-03-01');
16
17 -- Índices se crean por partición:
18 CREATE INDEX ON events_2025_01 (user_id);
19 CREATE INDEX ON events_2025_02 (user_id);
20
21 -- Partición default para fuera de rango:
22 CREATE TABLE events_default PARTITION OF events DEFAULT;
23
24 -- PG rutea automáticamente INSERT/UPDATE/DELETE a la partición
    correcta
25 INSERT INTO events (user_id, event_type, created_at)
26 VALUES (42, 'click', '2025-01-15');
27 -- Automáticamente va a events_2025_01

```

Listing 4.2: Partition pruning — verificar que funciona

```

1  -- Con EXPLAIN, verificar que el planner elide particiones
    irrelevantes:
2  EXPLAIN SELECT * FROM events
3  WHERE created_at BETWEEN '2025-01-01' AND '2025-01-31';
4
5  -- Debe aparecer solo events_2025_01, no todas las particiones
6  -- Si aparecen todas: revisar que el filtro use la columna de
    partición exacta
7
8  -- Manejar partition por hash (distribución uniforme sin semántica
    temporal):
9  CREATE TABLE users_sharded (id BIGSERIAL, email TEXT)
10 PARTITION BY HASH (id);
11
12 CREATE TABLE users_sharded_0 PARTITION OF users_sharded
13     FOR VALUES WITH (modulus 4, remainder 0);
14 CREATE TABLE users_sharded_1 PARTITION OF users_sharded
15     FOR VALUES WITH (modulus 4, remainder 1);
16 -- ... etc para remainder 2 y 3

```

4.3. 🏗️ Citus: Sharding sobre PostgreSQL

Citus es una extensión de PostgreSQL (ahora parte de Microsoft) que convierte PG en una BD distribuida. Un nodo coordinador recibe queries; las distribuye a workers que contienen fragmentos de datos.

🧠 Conceptos clave de Citus

- **Distributed table:** particionada en shards distribuidos entre workers por una *distribution column*
- **Reference table:** replicada en todos los workers (para tablas pequeñas en

- joins frecuentes: catálogos, configuración)
- **Local table:** solo en el coordinador (metadatos del sistema)
 - **Colocation:** shards de dos tablas con la misma distribution column en el mismo worker → joins locales, sin red

Listing 4.3: Setup básico de Citus

```

1  -- Instalar extensión:
2  CREATE EXTENSION citus;
3
4  -- Agregar workers al coordinador:
5  SELECT citus_add_node('worker1', 5432);
6  SELECT citus_add_node('worker2', 5432);
7
8  -- Distribuir tabla por tenant_id (SaaS típico):
9  SELECT create_distributed_table('orders', 'tenant_id');
10 SELECT create_distributed_table('items', 'tenant_id'); --
    colocated con orders
11
12 -- Reference table (catálogo de productos: misma en todos los
    workers):
13 SELECT create_reference_table('products');
14
15 -- Queries single-tenant van directo al shard correcto (rápido):
16 SELECT * FROM orders WHERE tenant_id = 42;
17
18 -- Queries cross-tenant se ejecutan en paralelo en todos los
    workers:
19 SELECT tenant_id, COUNT(*) FROM orders GROUP BY tenant_id;

```

4.4. Foreign Data Wrappers (FDW)

FDW permite acceder a fuentes externas (otra base PG, MySQL, S3, CSV) como si fueran tablas locales. Útil para federación de datos y migraciones incrementales.

Listing 4.4: postgres_fdw — acceso a otra base PG

```

1  -- En el servidor local:
2  CREATE EXTENSION postgres_fdw;
3
4  CREATE SERVER remote_orders
5  FOREIGN DATA WRAPPER postgres_fdw
6  OPTIONS (host 'remote-pg.example.com', port '5432', dbname '
    orders_db');
7
8  CREATE USER MAPPING FOR current_user
9  SERVER remote_orders
10 OPTIONS (user 'readonly_user', password 'secret');
11
12 -- Importar todas las tablas de un schema remoto:

```

```
13 IMPORT FOREIGN SCHEMA public
14   FROM SERVER remote_orders
15   INTO local_remote_schema;
16
17 -- O crear tabla foránea individual:
18 CREATE FOREIGN TABLE remote_customers (
19   id      BIGINT,
20   email   TEXT,
21   name    TEXT
22 ) SERVER remote_orders OPTIONS (schema_name 'public', table_name '
    customers');
23
24 -- Usar como tabla local (push-down de filtros si es posible):
25 SELECT * FROM remote_customers WHERE id = 42;
```

4.5. CockroachDB y Vitess: alternativas

Comparativa de sistemas distribuidos

- **CockroachDB:** NewSQL compatible con PostgreSQL wire protocol. Distribuye datos automáticamente, transacciones distribuidas con Raft consensus. Operacional más simple que Citus pero overhead de latencia por consensus en cada write.
- **Vitess:** sharding para MySQL (usado por YouTube, Slack). Middleware: no modifica MySQL internamente. Maneja connection pooling, query routing, resharding online. Compatible con MySQL 8.
- **YugabyteDB:** open-source, compatible con PG y Cassandra API. DocDB storage engine + Raft. Bueno para geografía (multi-region).
- **PlanetScale:** Vitess as a service. Branching de schema (como git para la BD).

Preguntas típicas

1. ¿Cuándo tiene sentido sharding y cuándo es prematuro?
2. ¿Cuál es la diferencia entre partitioning y sharding?
3. En Citus, ¿qué es colocation y por qué importa para el rendimiento de joins?
4. ¿Qué es una reference table en Citus? ¿Cuándo la usarías?
5. Explica el problema de hot shard y cómo elegir una buena distribution column.
6. ¿Cuándo usarías CockroachDB en lugar de PostgreSQL + Patroni?

Capítulo 5

Migraciones en Producción

5.1. El problema de las migraciones en producción

Alterar una tabla con millones de filas en producción sin downtime es uno de los desafíos más subestimados del DBA Senior. Un `ALTER TABLE ADD COLUMN NOT NULL` en una tabla de 100M filas puede bloquear todas las escrituras por minutos.



Locks en DDL: `ALTER TABLE` toma un `AccessExclusiveLock` que bloquea *todas* las lecturas y escrituras mientras dura. Si hay una transacción larga corriendo, el `ALTER` espera. Y mientras espera, *las queries nuevas también esperan* (cola de locks). Un `ALTER` de 5 segundos puede provocar 500 queries encoladas que provocan timeout cascada y aparente caída total del servicio.

5.2. Expand-Contract Pattern

El patrón para migraciones sin downtime. Divide cada cambio en tres fases deployables independientemente.



Las 3 fases del Expand-Contract

1. **Expand (Add):** agregar la nueva estructura (columna, tabla, índice) sin eliminar la vieja. El código nuevo puede leer ambas; el código viejo sigue funcionando solo con la vieja.
2. **Migrate (Backfill):** llenar la nueva estructura con datos existentes. Opcionalmente: actualizar el código para escribir en ambas.
3. **Contract (Remove):** eliminar la estructura vieja una vez que todo el código usa la nueva.

Listing 5.1: Expand-Contract: renombrar columna (ejemplo real)

```
1 -- OBJETIVO: renombrar orders.user_id → orders.customer_id
2 -- Esto NO se hace con: ALTER TABLE orders RENAME COLUMN user_id TO
  customer_id
3 -- (rompería todo el código viejo instantáneamente)
4
5 -- FASE 1 - EXPAND: agregar nueva columna
6 ALTER TABLE orders ADD COLUMN customer_id BIGINT;
```



```

7  -- Agregar índice (concurrente, no bloquea):
8  CREATE INDEX CONCURRENTLY idx_orders_customer_id ON orders (
9      customer_id);
10
11 -- FASE 2 - BACKFILL: copiar datos en batches (NO en una sola query
12 )
13 -- Script de backfill por lotes:
14 DO $$
15 DECLARE
16     batch_size INT := 5000;
17     last_id BIGINT := 0;
18     max_id BIGINT;
19 BEGIN
20     SELECT MAX(id) INTO max_id FROM orders;
21     WHILE last_id < max_id LOOP
22         UPDATE orders
23         SET customer_id = user_id
24         WHERE id > last_id AND id <= last_id + batch_size
25             AND customer_id IS NULL;
26         last_id := last_id + batch_size;
27         PERFORM pg_sleep(0.1); -- throttle para no saturar I/O
28     END LOOP;
29 END $$;
30
31 -- Deploy del código que escribe en AMBAS columnas (user_id Y
32 customer_id)
33
34 -- FASE 3 - CONTRACT: remover columna vieja
35 -- Primero asegurar que customer_id tiene NOT NULL (una vez
36 backfill completo):
37 ALTER TABLE orders ALTER COLUMN customer_id SET NOT NULL;
38 -- Luego eliminar la vieja:
39 ALTER TABLE orders DROP COLUMN user_id;
40 -- Eliminar índice viejo si existía:
41 DROP INDEX CONCURRENTLY idx_orders_user_id;

```

5.3. CREATE INDEX CONCURRENTLY

Listing 5.2: Diferencias entre index normal y CONCURRENTLY

```

1  -- Index normal: bloquea escrituras mientras construye
2  -- Tarda segundos a minutos dependiendo del tamaño
3  CREATE INDEX idx_orders_status ON orders (status);
4
5  -- CONCURRENTLY: no bloquea escrituras (solo reads afectados
6  brevemente al final)
7  -- Tarda más (2-3 pasadas), pero es seguro en producción
8  CREATE INDEX CONCURRENTLY idx_orders_status ON orders (status);
9
10 -- Si falla (crash, cancelación): queda en estado INVALID
11 -- Verificar:

```

```

11 SELECT indexname, indisvalid
12 FROM pg_indexes JOIN pg_index ON indexrelid = (
13     SELECT oid FROM pg_class WHERE relname = indexname
14 )
15 WHERE tablename = 'orders';
16
17 -- Limpiar index inválido:
18 DROP INDEX CONCURRENTLY idx_orders_status;
19 -- Luego recrear

```

5.4. pg_repack: reorganizar sin bloquear

VACUUM FULL compacta una tabla pero la bloquea completamente. pg_repack hace lo mismo construyendo la tabla nueva en segundo plano y haciendo swap atómico al final.

Listing 5.3: Uso de pg_repack

```

1 # Instalar extensión (en la BD):
2 # CREATE EXTENSION pg_repack;
3
4 # Reorganizar tabla (equivalente a VACUUM FULL sin lock prolongado)
5 :
6 pg_repack -h localhost -U postgres -d mydb -t orders
7
8 # Reorganizar tabla con nuevo orden físico (CLUSTER equivalente):
9 pg_repack -h localhost -U postgres -d mydb -t orders --order-by
10 created_at
11
12 # Solo índices (reorganizar bloat en índices):
13 pg_repack -h localhost -U postgres -d mydb -t orders --only-indexes
14
15 # Toda la base de datos:
16 pg_repack -h localhost -U postgres -d mydb

```

5.5. Flyway y Liquibase

Herramientas de versionado de esquema: registran qué migraciones se han aplicado y aplican las pendientes en orden.

Listing 5.4: Convenciones de Flyway

```

1 # Estructura de archivos:
2 # db/migration/
3 #   V1__init_schema.sql
4 #   V2__add_orders_table.sql
5 #   V3__add_customer_index.sql
6 #   R__create_reports_view.sql    (repeatable: se reaplica si cambia
7 )
8 # Naming: V{version}__{description}.sql

```

```

9 # Version puede ser numérica (1, 2, 3) o timestamp (20250115120000)
10
11 # Aplicar migraciones pendientes:
12 flyway migrate
13
14 # Ver estado:
15 flyway info
16
17 # Validar que migraciones aplicadas coinciden con archivos:
18 flyway validate
19
20 # Reparar tabla de migraciones (si hay migration fallida):
21 flyway repair

```

Listing 5.5: Liquibase con changesets XML

```

1 <!-- db/changelog/changelog.xml -->
2 <!-- <databaseChangeLog>
3   <changeSet id="1" author="aldo">
4     <createTable tableName="customers">
5       <column name="id" type="BIGINT" autoIncrement="true">
6         <constraints primaryKey="true"/>
7       </column>
8       <column name="email" type="VARCHAR(255)">
9         <constraints nullable="false" unique="true"/>
10      </column>
11    </createTable>
12  </changeSet>
13
14  <changeSet id="2" author="aldo">
15    <addColumn tableName="orders">
16      <column name="customer_id" type="BIGINT"/>
17    </addColumn>
18  </changeSet>
19 </databaseChangeLog> -->
20
21 # Aplicar:
22 liquibase update
23
24 # Rollback al changeset anterior:
25 liquibase rollback --tag=v1.0

```

Listing 5.6: Agregar NOT NULL sin downtime

```

1 -- Problema: ALTER TABLE t ALTER COLUMN c SET NOT NULL
2 -- escanea toda la tabla para verificar que no hay NULLs → lock
  largo
3
4 -- Solución moderna (PG 12+):
5 -- 1. Agregar constraint de check primero (sin NOT VALID → no
  escanea):
6 ALTER TABLE orders ADD CONSTRAINT orders_customer_id_not_null

```

```
7  CHECK (customer_id IS NOT NULL) NOT VALID;  
8  
9  -- 2. Validar en background (solo ShareUpdateExclusiveLock, no  
   bloquee):  
10 ALTER TABLE orders VALIDATE CONSTRAINT orders_customer_id_not_null;  
11  
12 -- 3. Ahora PG sabe que no hay NULLs → el SET NOT NULL es  
   instantáneo:  
13 ALTER TABLE orders ALTER COLUMN customer_id SET NOT NULL;  
14  
15 -- 4. Eliminar el constraint temporal:  
16 ALTER TABLE orders DROP CONSTRAINT orders_customer_id_not_null;
```



Preguntas típicas

1. Explica el Expand-Contract pattern. ¿Por qué no puedes simplemente renombrar una columna en un ALTER directo?
2. ¿Cuál es la diferencia entre CREATE INDEX y CREATE INDEX CONCURRENTLY? ¿Qué pasa si falla?
3. ¿Cómo agregas una columna NOT NULL a una tabla de 500M filas sin downtime?
4. ¿Qué pasa si una migración de Flyway falla a mitad? ¿Cómo lo resuelves?
5. ¿Cuándo usarías pg_repack vs VACUUM FULL?
6. ¿Qué es un “lock queue” en PostgreSQL y cómo un DDL puede tumbar tu servicio aunque tarde solo 2 segundos?

Capítulo 6



PITR, WAL y Backup

6.1. Write-Ahead Log en profundidad

WAL (Write-Ahead Log) es el mecanismo de durabilidad de PostgreSQL. **Regla fundamental:** ningún cambio se escribe al heap (datos) sin antes haberse escrito al WAL. Esto garantiza que tras un crash, el WAL puede re-aplicarse para recuperar el estado consistente.



Arquitectura del WAL

- **WAL segment:** archivos de 16MB en `pg_wal/`. Nombrados por LSN (Log Sequence Number): `000000010000000000000001`.
- **LSN (Log Sequence Number):** posición única en el flujo WAL. Formato: `X/YYYYZZZZ`. Siempre creciente.
- **Checkpoint:** evento periódico que garantiza que todas las páginas sucias hasta cierto LSN están escritas al heap. Permite reutilizar WAL anterior al checkpoint.
- **WAL writer:** proceso que flusha WAL buffers al disco. La frecuencia afecta latencia de commits.
- **WAL sender:** envía WAL a standbys (replicación) o al archivador.

Listing 6.1: Parámetros WAL críticos

```
1  -- postgresql.conf:
2  wal_level = replica           -- mínimo para replicación y archivado
3                                -- logical: replicación lógica también
4  checkpoint_timeout = 5min     -- checkpoint automático cada X (
    default 5min)
5  checkpoint_completion_target = 0.9 -- escribir páginas sucias en
    90% del intervalo
6  max_wal_size = 1GB           -- WAL puede crecer hasta X entre
    checkpoints
7  min_wal_size = 80MB          -- retener al menos X de WAL
8  wal_compression = on         -- comprimir WAL (reduce I/O a costo
    de CPU mínimo)
9
10 -- Ver LSN actual y estado de WAL:
11 SELECT pg_current_wal_lsn();   -- posición actual
12 SELECT pg_walfile_name(pg_current_wal_lsn()); -- nombre de archivo
    actual
```

```

13 SELECT pg_wal_lsn_diff(pg_current_wal_lsn(), '0/0'); -- bytes
    totales
14
15 -- Ver checkpoints recientes:
16 SELECT * FROM pg_stat_bgwriter;
17 -- checkpoints_timed: cuántos por timeout (bueno)
18 -- checkpoints_req: cuántos forzados por WAL lleno (malo → aumentar
    max_wal_size)

```

6.2. WAL Archiving

El archivado de WAL copia cada segmento completado a un almacenamiento externo (S3, NFS, etc.). Combinado con un base backup, permite PITR a cualquier punto en el tiempo.

Listing 6.2: Configurar WAL archiving

```

1 -- postgresql.conf:
2 archive_mode = on
3 archive_command = 'aws s3 cp %p s3://mi-bucket/wal-archive/%f'
4 -- %p = ruta completa del archivo WAL
5 -- %f = nombre del archivo (sin ruta)
6 -- El comando debe retornar 0 solo si la copia fue exitosa
7
8 -- Alternativa con rsync:
9 -- archive_command = 'rsync -a %p backup-server:/mnt/wal-archive/%f'
10
11 -- archive_cleanup_command: limpiar WAL archivados ya no necesarios
    (con pgBackRest):
12 -- archive_cleanup_command = 'pg_archivecleanup /mnt/wal-archive %r'
13
14 -- Verificar que el archivado funciona:
15 SELECT archived_count, last_archived_wal, last_archived_time,
16        failed_count, last_failed_wal, last_failed_time
17 FROM pg_stat_archiver;
18 -- failed_count > 0 y creciendo: problema con archive_command

```

6.3. pgBackRest: backup empresarial

pgBackRest es la herramienta de backup más completa para PostgreSQL: maneja base backups, WAL archiving, compresión, cifrado y PITR.

Listing 6.3: pgbackrest.conf — configuración básica

```

1 [global]
2 repo1-path=/var/lib/pgbackrest
3 repo1-retention-full=2           ; retener 2 full backups
4 repo1-retention-diff=7          ; retener 7 differential backups

```

```

5  repo1-cipher-type=aes-256-cbc
6  repo1-cipher-pass=supersecret
7
8  # Para S3:
9  # repo1-type=s3
10 # repo1-s3-bucket=mi-bucket-backups
11 # repo1-s3-region=us-east-1
12
13 [global:archive-push]
14 compress-level=3
15
16 [mydb]
17 pg1-path=/var/lib/postgresql/data
18 pg1-port=5432
19 pg1-user=postgres
20 \end{lstlisting}
21
22 \begin{lstlisting}[language=, caption={pgBackRest - operaciones
    comunes}]
23 # Inicializar el repositorio:
24 pgbackrest --stanza=mydb stanza-create
25
26 # Full backup:
27 pgbackrest --stanza=mydb --type=full backup
28
29 # Differential backup (desde último full):
30 pgbackrest --stanza=mydb --type=diff backup
31
32 # Incremental backup (desde último backup cualquiera):
33 pgbackrest --stanza=mydb --type=incr backup
34
35 # Ver información de backups:
36 pgbackrest --stanza=mydb info
37
38 # Restaurar último backup:
39 pgbackrest --stanza=mydb restore
40
41 # PITR: restaurar a punto específico en el tiempo:
42 pgbackrest --stanza=mydb restore \
43     --type=time "--target=2025-06-15 14:30:00"
44
45 # Verificar integridad del repositorio:
46 pgbackrest --stanza=mydb check

```

6.4. Procedimiento PITR

Point-In-Time Recovery: restaurar la base a cualquier momento entre el último base backup y el WAL archivado más reciente.

Listing 6.4: PITR manual con pg_restore + WAL

```

1 -- ESCENARIO: DROP TABLE accidental a las 14:23:00
2 -- Base backup más reciente: 02:00 AM
3 -- WAL archivado hasta: 14:30 (5 min después del desastre)
4
5 -- PASO 1: Detener PostgreSQL
6 -- systemctl stop postgresql
7
8 -- PASO 2: Mover datos actuales:
9 -- mv /var/lib/postgresql/data /var/lib/postgresql/data.broken
10
11 -- PASO 3: Restaurar base backup:
12 -- pgbackrest --stanza=mydb restore --delta (o desde tar manual)
13
14 -- PASO 4: Configurar recovery target en postgresql.conf:
15 -- (PG 12+: en el mismo postgresql.conf)
16 restore_command = 'pgbackrest --stanza=mydb archive-get %f "%p"'
17 recovery_target_time = '2025-06-15 14:22:59' -- un segundo antes
    del DROP
18 recovery_target_action = promote -- promote: cuando llegue al
    target, se convierte en primary
19
20 -- PASO 5: Crear señal de recovery:
21 -- touch /var/lib/postgresql/data/recovery.signal
22
23 -- PASO 6: Iniciar PostgreSQL:
24 -- systemctl start postgresql
25 -- PG aplica WAL hasta 14:22:59 y promueve
26
27 -- PASO 7: Verificar que la tabla existe y datos están correctos
28 -- PASO 8: Tomar nuevo base backup

```

Listing 6.5: RPO y RTO — métricas de recuperabilidad

```

1 -- RPO (Recovery Point Objective): ¿cuántos datos puedes perder?
2 -- Con archivado WAL cada ~60s: RPO ~= 60 segundos de datos
3 -- Con replicación síncrona: RPO = 0
4
5 -- RTO (Recovery Time Objective): ¿cuánto tiempo toma recuperarse?
6 -- Depende de: tamaño del backup + cantidad de WAL a replay
7 -- pgBackRest con backup diferencial puede reducir RTO vs full
    backup
8
9 -- Calcular WAL generado por día (útil para sizing de S3):
10 SELECT pg_size_pretty(SUM(size))
11 FROM pg_ls_waldir();
12
13 -- Probar recovery ANTES de necesitarla (no en producción):
14 -- Montar un servidor de prueba, restaurar, validar datos críticos
15 -- El backup que nunca probaste no es un backup

```




Preguntas típicas

1. ¿Qué es un LSN y cómo se relaciona con el WAL?
2. Explica la diferencia entre un checkpoint y un backup.
3. ¿Qué es PITR? ¿Qué necesitas para poder hacerlo?
4. Si `checkpoints_req` está creciendo en `pg_stat_bgwriter`, ¿qué indica y cómo lo corriges?
5. ¿Cuál es la diferencia entre RPO y RTO? Da un ejemplo concreto.
6. Un `DROP TABLE` ocurrió en producción hace 10 minutos. Tienes base backup y WAL archivado. Describe los pasos de recuperación.
7. ¿Por qué “el backup que nunca probaste no es un backup”?

Capítulo 7



Event Sourcing y CQRS

7.1. 📖 Event Sourcing: estado como historia

En un modelo tradicional, se guarda el **estado actual**. En Event Sourcing, se guarda la **historia de eventos**; el estado actual se deriva de ellos.



Comparativa: estado vs eventos

- **CRUD tradicional:** `UPDATE orders SET status='shipped'` — el estado anterior se pierde
- **Event Sourcing:** se inserta un evento `{type: OrderShipped, orderId: 42, at: ...}` — el estado anterior está implícito en la historia
- **Ventajas:** audit log gratuito, time travel (reconstruir estado en cualquier momento), debugging trivial, event replay para nuevas proyecciones
- **Desventajas:** complejidad mayor, eventual consistency en las vistas (read models), el estado actual requiere replay de todos los eventos (mitigado con snapshots)

7.2. 📖 Event Store con PostgreSQL

PostgreSQL es una base excelente para un event store: append-only, transaccional, con índices eficientes.

Listing 7.1: Schema de Event Store

```
1 -- Tabla principal de eventos:
2 CREATE TABLE events (
3     event_id          UUID          NOT NULL DEFAULT gen_random_uuid(),
4     stream_id         TEXT          NOT NULL,      -- ej: 'order-42', 'user
        -100'
5     stream_version    BIGINT        NOT NULL,      -- versión del stream (
        para OCC)
6     event_type        TEXT          NOT NULL,      -- 'OrderCreated', '
        OrderShipped', etc.
7     payload           JSONB         NOT NULL,
8     metadata          JSONB         NOT NULL DEFAULT '{}',
9     created_at        TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
10
11     PRIMARY KEY (event_id),
12     UNIQUE (stream_id, stream_version) -- evita conflictos de
        concurrencia
```

```

13 );
14
15 CREATE INDEX idx_events_stream ON events (stream_id, stream_version
16 );
17 CREATE INDEX idx_events_type ON events (event_type);
18 CREATE INDEX idx_events_created_at ON events (created_at);
19
20 -- Tabla de snapshots (para reconstrucción rápida):
21 CREATE TABLE snapshots (
22     stream_id      TEXT          NOT NULL,
23     stream_version BIGINT        NOT NULL,
24     state          JSONB         NOT NULL,
25     created_at     TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
26     PRIMARY KEY (stream_id, stream_version)
27 );

```

Listing 7.2: Append de eventos con Optimistic Concurrency Control

```

1  -- Agregar evento verificando versión esperada (OCC):
2  -- Si alguien más modificó el stream, la inserción falla por UNIQUE
   constraint
3  INSERT INTO events (stream_id, stream_version, event_type, payload)
4  VALUES (
5      'order-42',
6      5,    -- esperamos que la versión actual sea 4, insertamos la 5
7      'OrderShipped',
8      '{"carrier": "DHL", "tracking_code": "1Z999AA1"}'::jsonb
9  );
10 -- Si ya existe (stream_id, stream_version=5): ERROR duplicate key
11 -- La app debe hacer retry o reportar conflict
12
13 -- Obtener versión actual de un stream:
14 SELECT MAX(stream_version) FROM events WHERE stream_id = 'order-42'
15 ;
16
17 -- Reconstruir estado de un aggregate (replay):
18 SELECT event_type, payload, created_at
19 FROM events
20 WHERE stream_id = 'order-42'
21 ORDER BY stream_version ASC;
22
23 -- La app aplica cada evento al estado inicial vacío para obtener
   el estado actual

```

7.3. ✂ CQRS: separar lectura de escritura

CQRS (Command Query Responsibility Segregation) separa el modelo de escritura (Commands) del modelo de lectura (Queries/Projections).

CQRS con Event Store

- **Write side:** recibe Commands (`ShipOrder`, `CancelOrder`), valida, aplica lógica de negocio, emite Events al event store
- **Read side (Projections):** procesa events del store y actualiza vistas des-normalizadas optimizadas para lectura
- **Eventual consistency:** hay un delay entre que un evento se escribe y que la proyección lo refleja (generalmente milisegundos)
- Las proyecciones se pueden reconstruir completamente replaying todos los eventos desde el inicio

Listing 7.3: Tabla de proyección (read model)

```

1  -- Projection optimizada para UI de órdenes:
2  CREATE TABLE order_summary (
3      order_id          BIGINT          PRIMARY KEY,
4      customer_name     TEXT            NOT NULL,
5      status            TEXT            NOT NULL,
6      total_amount      NUMERIC         NOT NULL,
7      item_count        INT             NOT NULL DEFAULT 0,
8      last_event_at     TIMESTAMPTZ,
9      stream_version    BIGINT          NOT NULL -- checkpoint del último
        evento procesado
10 );
11
12 -- Actualizar projection al procesar un evento:
13 -- (esto corre en el projection worker, no en el request path)
14 INSERT INTO order_summary (order_id, customer_name, status,
        total_amount, stream_version)
15 VALUES (42, 'Ana García', 'created', 0, 1)
16 ON CONFLICT (order_id) DO UPDATE
17     SET status = EXCLUDED.status,
18         stream_version = EXCLUDED.stream_version,
19         last_event_at = NOW();

```

7.4. Outbox Pattern

Problema: guardar un evento en la BD Y publicarlo en Kafka en la misma operación. Si el mensaje a Kafka falla después de que la BD committeó, hay inconsistencia.

Solución: Outbox

1. La transacción de negocio escribe el evento en la BD y en una tabla outbox — todo en la misma transacción ACID.
2. Un proceso separado (poller o CDC) lee la tabla outbox y publica los mensajes a Kafka.
3. Marca los mensajes como procesados (o los elimina).
4. Si el publisher falla, los mensajes siguen en el outbox y se reintentarán.

At-least-once delivery.

Listing 7.4: Outbox table y proceso

```

1  -- Tabla outbox:
2  CREATE TABLE outbox (
3      id          UUID          NOT NULL DEFAULT gen_random_uuid()
4          PRIMARY KEY,
5      topic       TEXT          NOT NULL,      -- 'orders.events', '
          payments.events'
6      key        TEXT,          -- para particionamiento
          en Kafka
7      payload     JSONB         NOT NULL,
8      created_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
9      sent_at     TIMESTAMPTZ   -- NULL = pendiente
10 );
11
12 -- En la transacción de negocio:
13 BEGIN;
14 UPDATE orders SET status = 'shipped' WHERE id = 42;
15 INSERT INTO outbox (topic, key, payload)
16 VALUES ('orders.events', '42', '{"type":"OrderShipped","order_id":42} '::jsonb);
17 COMMIT;
18
19 -- Si el COMMIT falla → ambos rollbackean
20 -- Si el COMMIT exitoso → el outbox poller publicará el mensaje
21
22 -- El poller (puede ser pg_cron o un worker externo):
23 BEGIN;
24 SELECT id, topic, key, payload
25 FROM outbox WHERE sent_at IS NULL
26 ORDER BY created_at LIMIT 100
27 FOR UPDATE SKIP LOCKED;
28 -- Publicar a Kafka aquí...
29 UPDATE outbox SET sent_at = NOW() WHERE id IN (...);
30 COMMIT;

```

7.5. Replicación Lógica para CDC

Change Data Capture (CDC) captura cambios de la BD como un stream de eventos. La replicación lógica de PostgreSQL es una implementación nativa de CDC.

Listing 7.5: Replicación lógica como CDC

```

1  -- Requiere wal_level = logical en postgresql.conf
2
3  -- Crear publication (qué cambios capturar):
4  CREATE PUBLICATION orders_pub FOR TABLE orders, items;
5  -- O capturar todo:
6  CREATE PUBLICATION all_tables FOR ALL TABLES;
7

```

```
8 -- Crear subscription en otra base (que recibirá los cambios):
9 CREATE SUBSCRIPTION orders_sub
10 CONNECTION 'host=source_db port=5432 dbname=prod user=replicator
    password=secret'
11 PUBLICATION orders_pub;
12
13 -- Ver estado de la replicación lógica:
14 SELECT * FROM pg_stat_subscription;
15 SELECT * FROM pg_replication_slots;
16
17 -- Herramientas de CDC más maduras:
18 -- Debezium: captura WAL lógico → Kafka. Soporta PG, MySQL, MongoDB
19 -- pglogical: extensión para replicación lógica bidireccional entre
    PG
```



Preguntas típicas

1. ¿Cuál es la diferencia fundamental entre CRUD tradicional y Event Sourcing?
2. ¿Qué es un aggregate en el contexto de Event Sourcing?
3. Explica el problema que resuelve el Outbox Pattern. ¿Qué garantía de delivery ofrece?
4. ¿Qué es eventual consistency en CQRS? ¿Cómo lo manejas en la UI?
5. ¿Cuándo elegiría Event Sourcing sobre un modelo relacional clásico?
6. ¿Qué es CDC (Change Data Capture) y cómo lo implementarías en PostgreSQL?

Capítulo 8

Seguridad en PostgreSQL

8.1. Row-Level Security (RLS)

RLS permite que la BD misma filtre filas según el usuario actual, sin que la aplicación tenga que implementar los filtros. Ideal para arquitecturas multi-tenant donde cada tenant solo debe ver sus datos.

Cómo funciona RLS

- Habilitado por tabla con `ALTER TABLE ... ENABLE ROW LEVEL SECURITY`
- Las policies definen qué filas puede ver/modificar cada usuario o rol
- Un superuser o el owner de la tabla **no está sujeto a RLS** por defecto (a menos que se use `FORCE ROW LEVEL SECURITY`)
- Las policies se expresan como expresiones SQL que se agregan automáticamente a los `WHERE`

Listing 8.1: RLS para multi-tenant SaaS

```
1  -- Habilitar RLS en la tabla:
2  ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
3
4  -- Policy de SELECT: cada usuario solo ve sus propias órdenes
5  CREATE POLICY orders_tenant_isolation ON orders
6    FOR SELECT
7    USING (tenant_id = current_setting('app.current_tenant_id')::
8          BIGINT);
9
10 -- Policy de INSERT: solo puede insertar para su propio tenant
11 CREATE POLICY orders_tenant_insert ON orders
12   FOR INSERT
13   WITH CHECK (tenant_id = current_setting('app.current_tenant_id')
14             ::BIGINT);
15
16 -- Policy para UPDATE y DELETE:
17 CREATE POLICY orders_tenant_update ON orders
18   FOR UPDATE
19   USING (tenant_id = current_setting('app.current_tenant_id')::
20         BIGINT);
21
22 -- La app debe setear el tenant ID al inicio de cada sesión/
23   transacción:
24 SET app.current_tenant_id = '42';
```

```

21 -- O por transacción (más seguro):
22 BEGIN;
23     SET LOCAL app.current_tenant_id = '42';
24     SELECT * FROM orders; -- automáticamente filtrado a tenant 42
25 COMMIT;
26
27 -- Bypassar RLS para el admin (superuser ya bypassa por defecto):
28 ALTER TABLE orders FORCE ROW LEVEL SECURITY; -- aplica incluso al
    owner
29 -- Crear rol admin que bypassa:
30 CREATE ROLE admin_role BYPASSRLS;

```

Listing 8.2: Verificar políticas activas

```

1 SELECT schemaname, tablename, policyname, permissive, roles, cmd,
   qual
2 FROM pg_policies
3 WHERE tablename = 'orders';
4
5 -- Testear como usuario específico:
6 SET ROLE app_user;
7 SET app.current_tenant_id = '42';
8 SELECT COUNT(*) FROM orders; -- debe mostrar solo órdenes de
   tenant 42
9 RESET ROLE;

```

8.2. Cifrado con pgcrypto

Listing 8.3: Cifrado simétrico y hashing con pgcrypto

```

1 CREATE EXTENSION pgcrypto;
2
3 -- Hashing de contraseñas (bcrypt):
4 -- Guardar hash:
5 INSERT INTO users (email, password_hash)
6 VALUES ('user@example.com', crypt('mi_password_123', gen_salt('bf',
   12)));
7 -- 'bf' = blowfish (bcrypt), 12 = cost factor
8
9 -- Verificar contraseña:
10 SELECT id FROM users
11 WHERE email = 'user@example.com'
12     AND password_hash = crypt('mi_password_123', password_hash);
13 -- Si retorna fila → contraseña correcta
14
15 -- Cifrado simétrico de datos sensibles (AES):
16 -- Cifrar al insertar:
17 INSERT INTO payments (id, card_number_encrypted)
18 VALUES (1, pgp_sym_encrypt('4111111111111111', '
   clave_de_cifrado_secreta'));
19

```



```

20 -- Descifrar:
21 SELECT pgp_sym_decrypt(card_number_encrypted, '
    clave_de_cifrado_secreta')
22 FROM payments WHERE id = 1;
23
24 -- MEJOR PRÁCTICA: no guardar la clave en la query. Pasar desde app
    .
25 -- La clave debe venir del gestor de secretos (Vault, AWS Secrets
    Manager)
26 -- y nunca aparecer en los logs de queries
27
28 -- Cifrado asimétrico (PGP):
29 -- Generar claves con gpg externo, usar pgp_pub_encrypt /
    pgp_pub_decrypt

```

8.3. Autenticación y pg_hba.conf

Listing 8.4: pg_hba.conf — reglas de autenticación

```

1 # Formato: TYPE DATABASE USER ADDRESS METHOD
2 # Las reglas se evalúan de arriba hacia abajo; primera coincidencia
    aplica
3
4 # Rechazar conexiones locales de superuser por red:
5 hostssl all postgres 0.0.0.0/0 reject
6
7 # Requerir SSL + scram-sha-256 para conexiones de red:
8 hostssl all all 0.0.0.0/0 scram-sha-256
9
10 # Conexión local sin contraseña (para mantenimiento desde el
    servidor):
11 local all postgres peer
12
13 # Replicación solo desde servidor de standby específico:
14 hostssl replication replicator 10.0.0.2/32 scram-sha-256
15
16 # md5 está deprecado (vulnerable a MITM); usar scram-sha-256

```

Listing 8.5: SSL/TLS — requerir cifrado en tránsito

```

1 -- postgresql.conf:
2 ssl = on
3 ssl_cert_file = 'server.crt'
4 ssl_key_file = 'server.key'
5 ssl_ca_file = 'root.crt'      -- para verificación de clientes (mTLS
    )
6
7 -- Verificar que la conexión usa SSL:
8 SELECT ssl, version, cipher FROM pg_stat_ssl WHERE pid =
    pg_backend_pid();
9

```

```

10 -- Forzar SSL para un usuario:
11 ALTER USER app_user SET ssl = 'on'; -- no existe directamente;
    usar pg_hba.conf hostssl
12
13 -- En el cliente (psql):
14 -- psql "sslmode=verify-full sslrootcert=root.crt ..."

```

8.4. Principio de Mínimo Privilegio

Listing 8.6: Roles y privilegios granulares

```

1 -- Crear roles específicos por función:
2 CREATE ROLE app_readonly;
3 GRANT CONNECT ON DATABASE mydb TO app_readonly;
4 GRANT USAGE ON SCHEMA public TO app_readonly;
5 GRANT SELECT ON ALL TABLES IN SCHEMA public TO app_readonly;
6 ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO
    app_readonly;
7
8 CREATE ROLE app_writer;
9 GRANT app_readonly TO app_writer;
10 GRANT INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO
    app_writer;
11 ALTER DEFAULT PRIVILEGES IN SCHEMA public
12     GRANT INSERT, UPDATE, DELETE ON TABLES TO app_writer;
13
14 -- Usuario de la aplicación hereda del rol:
15 CREATE USER app_user WITH PASSWORD 'securepass';
16 GRANT app_writer TO app_user;
17
18 -- Nunca conectar la app como superuser o como el owner del schema
19 -- El owner puede DROP TABLE, ALTER TABLE, etc.
20
21 -- Revocar privilegios no necesarios:
22 REVOKE CREATE ON SCHEMA public FROM PUBLIC; -- PG 14+ lo hace por
    defecto
23 REVOKE ALL ON DATABASE mydb FROM PUBLIC;

```

8.5. pg_audit: logging de queries

Listing 8.7: Configurar pg_audit para compliance

```

1 # postgresql.conf:
2 shared_preload_libraries = 'pgaudit'
3 pgaudit.log = 'write, ddl' # ddl: CREATE/ALTER/DROP; write:
    INSERT/UPDATE/DELETE
4 pgaudit.log_catalog = off # no loggear queries de sistema (
    pg_catalog)
5 pgaudit.log_relation = on # incluir nombre de tabla/vista

```

```
6 pgaudit.log_parameter = on    # incluir parámetros de prepared
   statements
7
8 # Por rol específico:
9 # ALTER ROLE audited_user SET pgaudit.log = 'all';
10
11 # Formato del log:
12 # AUDIT: SESSION,1,1,DDL,CREATE TABLE,TABLE,public.users,...
13 # AUDIT: SESSION,2,1,WRITE,INSERT,TABLE,public.orders,...
```



Preguntas típicas

1. ¿Qué es Row-Level Security y cuándo la usarías?
2. Explica la diferencia entre `USING` y `WITH CHECK` en una policy de RLS.
3. ¿Por qué md5 está deprecado en `pg_hba.conf`? ¿Qué usar en su lugar?
4. Un desarrollador quiere conectar la app a PG como superuser “por simplicidad”. ¿Qué le dices?
5. ¿Cómo almacenarías contraseñas de usuarios en PostgreSQL? ¿Y números de tarjeta de crédito?
6. ¿Qué es mTLS y cuándo lo usarías en una conexión a PostgreSQL?

Capítulo 9

Monitoring y Observabilidad

9.1. 🔍 pg_stat_statements: queries más costosas

La extensión más importante para performance tuning en producción. Registra estadísticas de cada query única (normalizada), incluyendo tiempo total, llamadas y varianza.

Listing 9.1: Setup y uso de pg_stat_statements

```
1  -- postgresql.conf:
2  shared_preload_libraries = 'pg_stat_statements'
3  pg_stat_statements.max = 10000          -- queries distintas a
    retener
4  pg_stat_statements.track = all          -- top, all, none
5  pg_stat_statements.track_io_timing = on -- medir tiempo de I/O (
    útil, overhead bajo)
6
7  -- Habilitar la extensión en la BD:
8  CREATE EXTENSION pg_stat_statements;
9
10 -- TOP 10 queries por tiempo total:
11 SELECT
12     round(total_exec_time::numeric, 2) AS total_ms,
13     calls,
14     round(mean_exec_time::numeric, 2) AS mean_ms,
15     round(stddev_exec_time::numeric, 2) AS stddev_ms,
16     round((total_exec_time / SUM(total_exec_time) OVER ()) * 100, 2)
        AS pct_total,
17     left(query, 80) AS query_snippet
18 FROM pg_stat_statements
19 ORDER BY total_exec_time DESC
20 LIMIT 10;
21
22 -- TOP 10 por llamadas (alta frecuencia):
23 SELECT calls, round(mean_exec_time::numeric, 2) AS mean_ms,
24         left(query, 80) AS query
25 FROM pg_stat_statements
26 ORDER BY calls DESC LIMIT 10;
27
28 -- Queries con alta varianza (inconsistentes):
29 SELECT round(stddev_exec_time::numeric, 2) AS stddev_ms,
30         round(mean_exec_time::numeric, 2) AS mean_ms,
31         left(query, 80) AS query
32 FROM pg_stat_statements
```

```

33 WHERE calls > 100
34 ORDER BY stddev_exec_time DESC LIMIT 10;
35
36 -- Resetear estadísticas:
37 SELECT pg_stat_statements_reset();

```

9.2. Vistas de sistema esenciales

Listing 9.2: Queries críticas de monitoreo

```

1  -- Conexiones activas y sus queries:
2  SELECT pid, username, application_name, client_addr, state,
3         now() - query_start AS duration,
4         wait_event_type, wait_event,
5         left(query, 100) AS query
6  FROM pg_stat_activity
7  WHERE state != 'idle'
8  ORDER BY duration DESC;
9
10 -- Detectar locks y bloqueos:
11 SELECT blocked_locks.pid AS blocked_pid,
12        blocked_activity.query AS blocked_query,
13        blocking_locks.pid AS blocking_pid,
14        blocking_activity.query AS blocking_query,
15        now() - blocked_activity.query_start AS wait_time
16  FROM pg_catalog.pg_locks blocked_locks
17  JOIN pg_catalog.pg_stat_activity blocked_activity ON
18        blocked_activity.pid = blocked_locks.pid
19  JOIN pg_catalog.pg_locks blocking_locks
20    ON blocking_locks.locktype = blocked_locks.locktype
21     AND blocking_locks.relation = blocked_locks.relation
22     AND blocking_locks.granted = true
23     AND blocked_locks.granted = false
24  JOIN pg_catalog.pg_stat_activity blocking_activity ON
25        blocking_activity.pid = blocking_locks.pid
26  WHERE blocked_locks.NOT granted;
27
28 -- Tamaño de tablas e índices:
29 SELECT schemaname, relname,
30        pg_size_pretty(pg_total_relation_size(oid)) AS total_size,
31        pg_size_pretty(pg_relation_size(oid)) AS table_size,
32        pg_size_pretty(pg_total_relation_size(oid) -
33        pg_relation_size(oid)) AS index_size
34  FROM pg_class
35  WHERE relkind = 'r'
36  ORDER BY pg_total_relation_size(oid) DESC
37  LIMIT 20;
38
39 -- Hit rate de shared_buffers (debe ser > 99%):
40 SELECT
41     sum(heap_blks_hit) AS heap_hit,

```

```

39  sum(heap_blks_read) AS heap_read,
40  round(sum(heap_blks_hit)::numeric /
41        NULLIF(sum(heap_blks_hit) + sum(heap_blks_read), 0) * 100,
42        2) AS cache_hit_pct
FROM pg_statio_user_tables;

```

9.3. pgBadger: análisis de logs

pgBadger analiza los logs de PostgreSQL y genera reportes HTML con las queries más lentas, los errores más frecuentes y métricas de conexiones.

Listing 9.3: Configurar logs para pgBadger

```

1  # postgresql.conf:
2  log_destination = 'csvlog'           # o 'stderr' con
   log_line_prefix
3  log_min_duration_statement = 1000    # loggear queries > 1 segundo
4  log_line_prefix = '%t [%p]: [%l-1] user=%u,db=%d,app=%a,client=%h '
5  log_checkpoints = on
6  log_connections = on
7  log_disconnections = on
8  log_lock_waits = on
9  log_temp_files = 0                  # loggear cuando se usan temp
   files (indica work_mem bajo)
10 log_autovacuum_min_duration = 0      # loggear todos los autovacuum

```

Listing 9.4: Generar reporte con pgBadger

```

1  # Analizar log del día de hoy:
2  pgbadger /var/log/postgresql/postgresql-2025-06-15.log -o report.
   html
3
4  # Modo incremental (para crons diarios):
5  pgbadger /var/log/postgresql/*.log --incremental --outdir /var/www/
   pgbadger/
6
7  # Filtrar solo queries lentas (> 5 segundos):
8  pgbadger /var/log/postgresql/postgresql.log \
9  --slow-query-only --threshold 5000 -o slow_queries.html

```

9.4. Prometheus + postgres_exporter

Listing 9.5: postgres_exporter — métricas para Prometheus

```

1  # docker-compose.yml:
2  # services:
3  #   postgres_exporter:
4  #     image: quay.io/prometheuscommunity/postgres-exporter
5  #     environment:

```

```

6 # DATA_SOURCE_NAME: "postgresql://exporter:password@postgres
  :5432/mydb?sslmode=require"
7 # ports:
8 #   - "9187:9187"
9 # volumes:
10 #   - ./queries.yaml:/etc/postgres_exporter/queries.yaml
11
12 # Métricas expuestas por defecto:
13 # pg_up - 1 si PG está arriba
14 # pg_database_size_bytes - tamaño por BD
15 # pg_stat_user_tables_n_dead_tup - dead tuples por tabla
16 # pg_stat_statements_total_time_seconds - tiempo total por query (
  si habilitado)
17 # pg_replication_lag_seconds - lag de replicación
18 # pg_locks_count - locks por tipo

```

Listing 9.6: Alertas críticas en Prometheus (alertmanager rules)

```

1 # prometheus/rules/postgres.yml:
2 # groups:
3 # - name: postgres
4 #   rules:
5 #     - alert: PostgresReplicationLagHigh
6 #       expr: pg_replication_lag_seconds > 30
7 #       for: 5m
8 #       labels:
9 #         severity: critical
10 #       annotations:
11 #         summary: "Replication lag > 30s on {{ $labels.instance }}"
12 #
13 #     - alert: PostgresCacheHitRateLow
14 #       expr: pg_stat_database_blks_hit /
15 #         (pg_stat_database_blks_hit + pg_stat_database_blks_read
16 #         ) < 0.95
17 #       for: 10m
18 #       labels:
19 #         severity: warning
20 #
21 #     - alert: PostgresDeadTuplesHigh
22 #       expr: pg_stat_user_tables_n_dead_tup > 1000000
23 #       for: 30m
24 #       labels:
25 #         severity: warning
26 #
27 #     - alert: PostgresConnectionsHigh
28 #       expr: pg_stat_activity_count / pg_settings_max_connections >
29 #         0.8
30 #       for: 5m
31 #       labels:
32 #         severity: critical

```

9.5. Dashboards Grafana esenciales



Métricas clave para un dashboard de PostgreSQL

- **Conexiones:** total activas vs `max_connections`. Alerta a 80 %.
- **Cache hit rate:** `shared_buffers` efectividad. Debe ser >99 %.
- **Replication lag:** en bytes (write/flush/replay lag) y en tiempo.
- **Dead tuples:** por tabla. Pico = autovacuum no alcanza el ritmo de cambios.
- **Queries/segundo:** por tipo (SELECT/INSERT/UPDATE/DELETE) — desde `pg_stat_database`.
- **Lock waits:** tiempo de espera por locks. Spikes = contención.
- **Checkpoints:** `checkpoints_req` vs `checkpoints_timed`. Req alto = `max_wal_size` pequeño.
- **Temp files:** bytes escritos a disco por sorts. Alto = `work_mem` insuficiente.



Preguntas típicas

1. ¿Qué es `pg_stat_statements`? ¿Qué overhead tiene y por qué vale la pena?
2. La cache hit rate cayó de 99.5 % a 92 %. ¿Qué puede haberlo causado y cómo investigas?
3. ¿Cómo detectas un deadlock en PostgreSQL en tiempo real?
4. ¿Qué indican los temp files en los logs de PostgreSQL y cómo los eliminas?
5. Diseña un sistema de alertas para PostgreSQL en producción. ¿Cuáles son las 5 alertas más críticas?
6. ¿Qué hace pgBadger y cuándo lo usarías vs `pg_stat_statements`?

Capítulo 10

Preguntas de Entrevista Senior/Staff

Este capítulo reúne las preguntas más difíciles y representativas de entrevistas para roles DBA Senior, Staff Engineer o Database Engineer. Las respuestas son intencionalmente abiertas: buscan razonamiento, no memorización.

10.1. Diseño de sistemas



Preguntas típicas

1. **(Diseño)** Diseña el schema de base de datos para un sistema de mensajería estilo WhatsApp con 1B usuarios. Considera: conversaciones grupales, mensajes, estados de lectura (doble check), búsqueda de mensajes, y archivos adjuntos.
2. **(Escala)** Tienes una tabla `orders` con 2B filas que crece 10M filas/día. Las queries más frecuentes filtran por `created_at` (últimos 90 días) y por `customer_id`. El autovacuum no alcanza. ¿Cuál es tu plan de acción de corto y largo plazo?
3. **(Arquitectura)** Tu equipo propone migrar de un monolito PostgreSQL a un sistema de microservicios con una BD por servicio. ¿Qué riesgos de datos identificas? ¿Cómo manejas las transacciones que cruzan servicios? ¿Cuándo esta migración tiene sentido vs cuándo es prematura?
4. **(Multi-tenant)** Diseña la estrategia de aislamiento de datos para un SaaS B2B con 5000 tenants de tamaños muy distintos (desde 10 usuarios hasta 100,000). Compara: BD por tenant, schema por tenant, RLS en schema compartido.
5. **(Tiempo real)** Diseña un sistema de leaderboard global en tiempo real para un juego con 50M usuarios activos simultáneos. ¿Qué BD usarías para los scores? ¿Para el ranking? ¿Cómo manejas los empates y la consistencia del ranking?

10.2. ⚡ Performance y troubleshooting

Preguntas típicas

1. **(Incident)** A las 3am recibes una alerta: las queries a la BD están tardando 10x más que normal. El servidor tiene CPU al 15 %, RAM al 60 %, I/O al 90 %. ¿Cuáles son tus primeros 5 pasos de diagnóstico? ¿Qué herramientas usas?
2. **(Query)** Tienes esta query que tarda 45 segundos y debes bajarla a <1 segundo:

```
SELECT c.name, SUM(o.total) FROM customers c LEFT JOIN orders o ON o.customer_id = c.id WHERE c.country = 'MX' AND o.created_at > NOW() - INTERVAL '1 year' GROUP BY c.name ORDER BY SUM(o.total) DESC;
```

 ¿Cómo la analizas? ¿Qué índices considerarías? ¿Qué otros cambios posibles hay?
3. **(Locking)** Una migración de 5 segundos (`ALTER TABLE ADD COLUMN`) causó 2 minutos de downtime completo. Explica exactamente qué ocurrió y cómo lo evitas en el futuro.
4. **(Autovacuum)** `pg_stat_user_tables` muestra 50M dead tuples en tu tabla de eventos después de un batch update masivo. Autovacuum sigue corriendo pero el número no baja. ¿Qué puede estar causando esto y cómo lo resuelves?
5. **(Memory)** Tus sorts con `ORDER BY` están usando 2GB de temp files en disco (visible en logs). `work_mem` está en 16MB. Quieres subirlo pero tienes 200 conexiones activas. ¿Cómo calculas el valor seguro y cómo lo aplicas sin riesgo de OOM?

10.3. 🔄 High Availability y recuperación

Preguntas típicas

1. **(Failover)** Tu Patroni primary muere a las 2pm. El standby tiene 3 minutos de lag de replicación. ¿Qué pasa automáticamente? ¿Qué pasa con las transacciones que estaban en vuelo? ¿Qué datos se pierden? ¿Cómo notificas a los usuarios?
2. **(Recovery)** Un DBA junior ejecutó `DELETE FROM users` sin `WHERE` en producción a las 11:47am. Tienes base backup de las 2am y WAL archivado completo. Describe paso a paso el proceso de PITR. ¿Qué pasa con los datos escritos entre las 2am y las 11:47am?
3. **(Split-brain)** Explica el escenario de split-brain en un clúster de PG. ¿Cómo lo detecta Patroni? ¿Qué sacrifica el sistema para prevenirlo? (Hint: CAP theorem)
4. **(RPO/RTO)** Tu equipo tiene `RPO = 5 minutos` y `RTO = 30 minutos`. Diseña la arquitectura de backup y recuperación para PostgreSQL que cumpla estos SLAs. ¿Qué monitoreo implementas para verificar que el RPO

se mantiene?

5. **(Geo)** Tus usuarios están en México, Europa y Asia. El P99 de latencia de BD es 800ms para Europa y Asia. ¿Qué estrategias de distribución considerarías? ¿Qué cambios de aplicación requieren?

10.4. 🧠 Preguntas de fundamentos a nivel Staff

🗣️ Preguntas típicas

1. **(MVCC)** Explica el modelo MVCC de PostgreSQL. ¿Por qué un UPDATE en PG es más costoso que en Oracle? ¿Cómo impacta esto en el diseño de tablas con alta tasa de updates?
2. **(Transactions)** Explica las anomalías de concurrencia: dirty read, non-repeatable read, phantom read y serialization anomaly. ¿Qué nivel de aislamiento protege contra cada una? ¿Cuándo usarías SERIALIZABLE en producción?
3. **(Índices)** Explica internamente cómo funciona un índice B-tree. ¿Por qué es eficiente para range scans? ¿Cuándo un índice hash sería mejor? ¿Qué es un GIN index y para qué tipos de columnas aplica?
4. **(CAP)** Explica el CAP theorem en el contexto de una BD distribuida. ¿Dónde ubicas PostgreSQL en ese espectro? ¿Y CockroachDB? ¿Qué significa “CP” para las garantías que ofreces a tus usuarios?
5. **(Schema evolution)** Tu equipo trabaja con trunk-based development y deploy continuo. Un deploy incluye un cambio de schema (agregar columna NOT NULL con default). ¿Cómo garantizas que el deploy sea backward compatible con la versión anterior del código aún corriendo durante el rollout?

10.5. 📌 Cheat sheet de respuestas clave

🧠 Números que debes saber de memoria

- **Página de PG:** 8KB por defecto
- **Segmento WAL:** 16MB por defecto
- **max_connections default:** 100
- **shared_buffers recomendado:** 25 % de RAM
- **effective_cache_size recomendado:** 75 % de RAM
- **random_page_cost default:** 4.0 (HDD); ajustar a 1.1–1.5 en SSD
- **statistics target default:** 100 (rango: 1–10000)
- **checkpoint_timeout default:** 5 minutos
- **autovacuum_vacuum_scale_factor default:** 0.2 (20 % de filas muer-

tas)

- **WAL level mínimo para replicación:** replica
- **WAL level para replicación lógica:** logical



Patrones de diseño que debes dominar

- **Expand-Contract:** migraciones sin downtime
- **Outbox Pattern:** at-least-once delivery entre BD y message broker
- **Read Replica Pattern:** separar carga de lectura/escritura
- **Shard by Tenant:** multi-tenant con Citus o partitioning
- **Event Sourcing + CQRS:** sistemas de auditoría y alta consistencia
- **Saga Pattern:** transacciones distribuidas en microservicios
- **Circuit Breaker:** proteger la BD de cascading failures